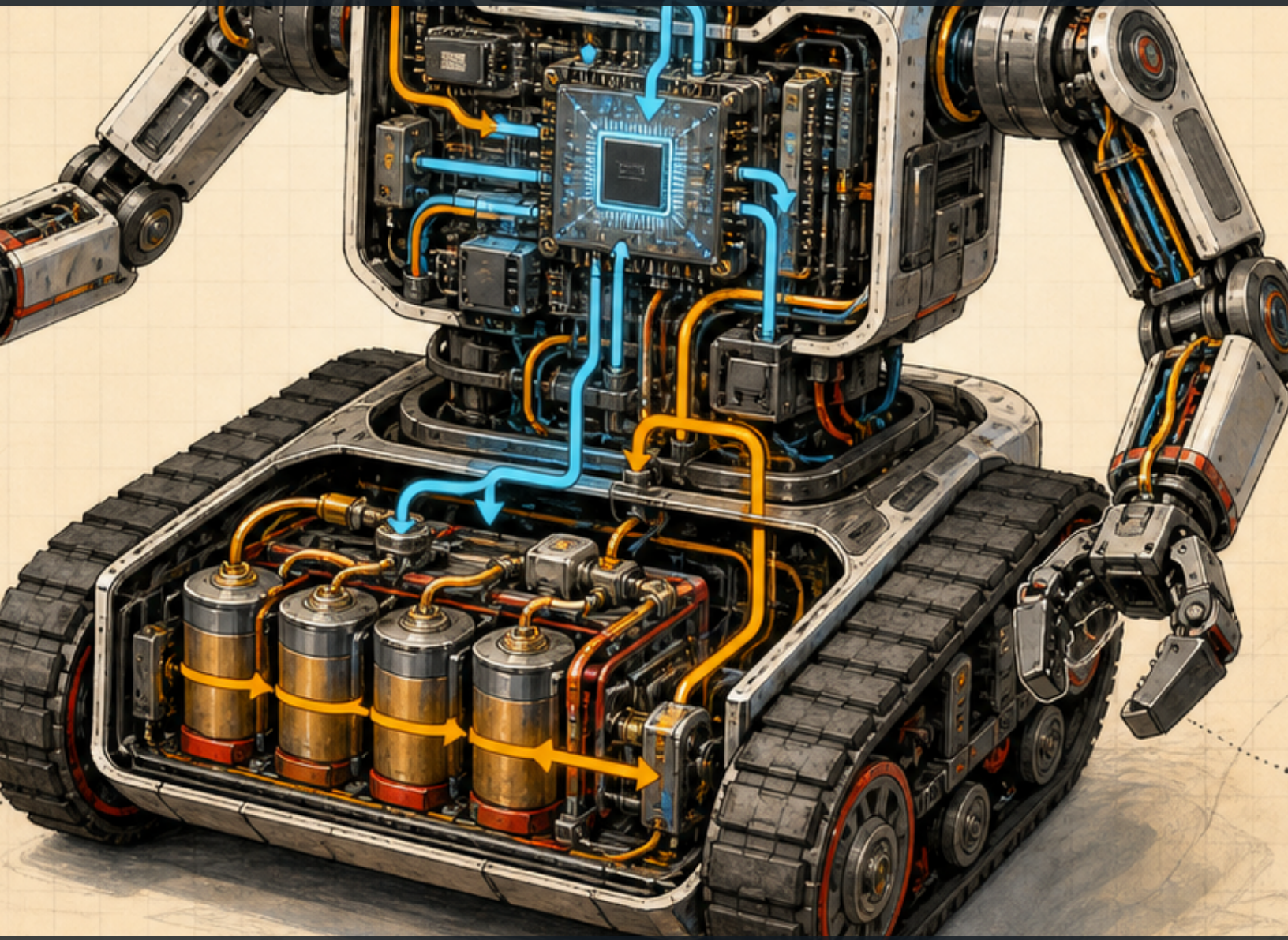

BUILDING R.O.B. - VOLUME 7

ENGINEERING ROBCon- trollerVision

Swift 6, spatial input, secure transport, H.264 video, and speech



AN IMPLEMENTATION-LEVEL GUIDE FOR SWIFT DEVELOPERS

Rodolfo Aramayo / Orbitus Robotics - First Maker Faire Edition

Write the controller as a safety-critical distributed client

A beautiful spatial interface still needs strict state, time, identity, and ownership boundaries.

This volume reads the current ROBControllerVision source as a serious Swift codebase. It explains why types live in the application or package, how actor isolation and structured concurrency protect lifecycle state, how controller and head pose become leased demands, how authenticated control and video remain separate, how H.264 reaches an AVSampleBufferDisplayLayer, and how explicit microphone capture becomes bounded text rather than raw audio streaming.

Paths are relative to the inspected workspace. Every chapter names the exact source files being analyzed. Code changes after this edition must be checked against the repository, tests, SDK documentation, and physical robot behavior.

Copyright © 2026 Rodolfo Aramayo / Orbitus Robotics. Photographs are from the private ROB build archive unless otherwise noted. The generated frontispiece is original artwork derived from a ROB reference photograph. This independent educational project is not affiliated with or endorsed by any film studio or franchise owner. Product and company names remain the property of their respective owners.

First evidence-based draft: 2 August 2026. Build details marked **Builder input needed** are deliberate placeholders for the builder to complete.

A VISION PRO APP IS NOT THE ROBOT'S FINAL SAFETY LAYER

Client-side arming, dead-man evaluation, leases, and emergency-stop commands are valuable but cannot stop hardware after every possible network, process, power, or actuator failure. Cerebro and lower layers must reject stale input and stop independently. A verified physical emergency stop and controlled test procedure remain mandatory.

The Building R.O.B. library

VOLUME 1	<i>Meet ROB: A Robot Is a Team of Systems</i> - ages 8-12
VOLUME 2	<i>Circuits and Signals with ROB</i> - ages 10-14
VOLUME 3	<i>Motion Workshop: Treads, Gears, and Fabrication</i> - ages 10-15
VOLUME 4	<i>Mission Control: Arduino, Mac, Networks, and Vision</i> - ages 12-16
VOLUME 5	<i>AI, Robotics, and Codex: Directing, Verifying, and Owning AI-Built Systems</i> - advanced makers
VOLUME 6	<i>Dual-Arm Robotics: AMBER B1, URDF, CAN, and Ubuntu</i> - advanced makers
VOLUME 7	<i>Engineering ROBControllerVision: Swift, Spatial Input, Video, and Speech</i> - Swift developers
MANUAL	<i>Building R.O.B.: The Complete Maker's Field Manual</i> - advanced builders

HOW TO USE THIS BOOK

This is a code-reading and implementation book. Keep the repository open beside the PDF, run package tests after each domain change, compile the visionOS target, and use the simulator before attempting a two-device or powered-hardware test.

HOW TO FIND THE FILES

Every printed path is relative to the shared workspace root, `/Users/rob/dev` in the inspected edition. Start at that root, enter the first directory in the path, and follow each slash. Source code is spread across independent repositories—including `ROBArduino`, `Cerebro`, `ROBController`, `ROBControllerVision`, `ORobotics`, and `ROBTrainingGames`—rather than one giant ROB project. See `Presentation/ROB-Books/OPEN-SOURCE-CODE-MAP.md` for repository locations, licenses, primary files, and commands for finding renamed files.

Contents

- 1 Read this book with the repository open** **1**
- 2 Start with the build graph** **2**
 - 2.1 Reproduce the build 2
- 3 Compose the application at one obvious boundary** **3**
- 4 Use Observation without turning everything into UI state** **4**
- 5 Make lifecycle cancellation explicit** **5**
- 6 Put the state machine in an actor** **6**
- 7 Define a domain protocol before a wire format** **7**
- 8 Treat dead-man logic as a pure decision function** **8**
- 9 Translate controller hardware into one sample** **9**
 - 9.1 Combine two controllers carefully 9
- 10 Convert tank drive without losing information** **10**
- 11 Track head orientation relative to consent** **11**
- 12 Understand controller pose as telemetry, not authority** **12**
- 13 Design press-and-hold spatial controls** **13**
- 14 Pair identity, not merely an address** **14**
- 15 Separate control and video failure domains** **15**
- 16 Negotiate video before allocating a decoder** **16**
- 17 Frame ordered QUIC video defensively** **17**
- 18 Carry codec configuration and access units explicitly** **18**
- 19 Decode without blocking control or the main actor** **19**
- 20 Present AVFoundation media inside SwiftUI** **20**
- 21 Understand audio: capture locally, transmit text** **21**
 - 21.1 Command versus puppet speech 21
 - 21.2 If true audio streaming is added later 21

22 Declare privacy and device capabilities	22
23 Design failures as ordinary states	23
24 Test the package in layers	24
25 Debug without drowning Xcode	25
26 Add a feature without breaking authority	26
27 Swift techniques worth carrying elsewhere	27
28 Production review checklist	28
29 File-by-file reading order	29
30 Closing principle	30

Read this book with the repository open

ROBControllerVision is a native SwiftUI visionOS controller with a reusable Swift package. It connects to the real Cerebro application through authenticated Network.framework transports or to a deterministic simulator. The source root for this volume is:

```
1 ROBControllerVision/
```

The application is under `ROBControllerVision/ROBControllerVision`. Reusable domain, video, and Cerebro adapter code is under `ROBControllerVision/Packages/ROBControlCore`. Tests live beside that package. Matching robot-side behavior is in the separate Cerebro repository.

SOURCE TRAIL – ANALYZING NOW: `ROBControllerVision/README.md`, `docs/architecture.md`, `docs/protocol-v2.md`, and `docs/real-video-integration.md` describe the intended system. Source and tests decide what the current build actually does.

This book uses **observed** for behavior directly supported by the inspected source, **design requirement** for an invariant the code intends to preserve, and **proposed** for an extension not yet implemented.

Start with the build graph

SOURCE TRAIL – ANALYZING NOW: the Xcode project's `project.pbxproj` and the local package's `Package.swift`. The source map gives their full paths.

The Xcode application target owns visionOS lifecycle, SwiftUI presentation, ARKit, GameController, speech permission, and the app-facing view model. The Swift package declares three libraries:

- **ROBControlCore** contains pure domain types, session state, leases, dead-man logic, simulator, and video-domain contracts.
- **ROBCerebroTransport** contains `Network.framework`, `Security.framework`, pairing, discovery, pinned TLS, legacy control compatibility, and the separate video client.
- **ROBVideoPipeline** contains bounded media channels, H.264 support, validation, sample-buffer construction, and synthetic video.

This dependency direction is deliberate. Domain code must not import SwiftUI, GameController, ARKit, or a concrete network adapter. The app may import all three products. The transport imports the domain contract. The video pipeline imports the video-domain types.

The package requires Swift tools 6.0 and supports iOS 18, macOS 15, and visionOS 2. Swift 6 concurrency checking therefore belongs to the architecture, not merely compiler polish.

Reproduce the build

```
1 swift test --package-path Packages/ROBControlCore
2
3 xcodebuild \
4   -project ROBControllerVision.xcodeproj \
5   -scheme ROBControllerVision \
6   -configuration Debug \
7   -destination 'generic/platform=visionOS Simulator' \
8   -derivedDataPath /tmp/ROBControllerVisionDerivedData \
9   CODE_SIGNING_ALLOWED=NO build
```

Change the placeholder bundle identifier and select a development team only when preparing a physical-device build. Do not commit a personal signing identity.

Compose the application at one obvious boundary

SOURCE TRAIL – ANALYZING NOW: App/ROBControllerVisionApp.swift, App/ContentView.swift, and App/RobotViewModel.swift.

The app entry creates long-lived application state. ContentView receives a bindable model and composes three fixed cockpit regions in an HStack: controls on the left, a large camera surface in the center, and connection, speech, telemetry, and safety information on the right.

The layout intentionally has no vertical ScrollView. The window is 1760 by 920 points; the center camera is 920 points wide and each side wing is 360 by 820. Side panels receive an eight-degree Y-axis rotation3DEffect, a 42-point Z offset, perspective 0.22, and a shadow. This produces depth without placing safety controls in a fully immersive scene.

The central SwiftUI pattern is:

```

1 struct ContentView: View {
2     @Bindable var model: RobotViewModel
3     @FocusState private var receivesControllerEvents: Bool
4
5     var body: some View {
6         HStack { /* left controls, video, right diagnostics */ }
7             .handlesGameControllerEvents(matching: .gamepad)
8             .focusable()
9             .focused($receivesControllerEvents)
10            .task {
11                receivesControllerEvents = true
12                model.start()
13            }
14            .onDisappear { model.stop() }
15    }
16 }
```

@Bindable projects bindings from an Observation model. @FocusState is operational: GameController events depend on the view receiving focus. The .task modifier couples asynchronous startup to view presence, while onDisappear forces teardown.

Do not hide process ownership in many views. RobotViewModel is the application coordinator; feature views render state and invoke named intents.

Use Observation without turning everything into UI state

SOURCE TRAIL – ANALYZING NOW: App/RobotViewModel.swift.

RobotViewModel is @MainActor, @Observable, and final. That combination communicates three decisions:

1. UI-observable mutation occurs on the main actor.
2. SwiftUI can track properties without ObservableObject and @Published boilerplate.
3. The class is not designed for inheritance.

Properties used for rendering—snapshot, status messages, selected endpoint, speed, text draft—remain observable. Internal tasks, the session actor, simulator, pairing store, lifecycle IDs, and cached input samples use @ObservationIgnored. This prevents observation tracking from becoming an accidental synchronization mechanism.

The initializer performs dependency injection:

```
1 init(  
2     session: RobotSession = RobotSession(),  
3     simulator: SimulatedRobotEndpoint? = nil,  
4     videoPipeline: VideoPipelineCoordinator = VideoPipelineCoordinator(),  
5     pairingStore: ROBCerebroPairingStore = ROBCerebroPairingStore()  
6 )
```

Defaults keep production composition easy. Parameters let tests substitute controlled dependencies. The simulator default receives a SyntheticVideoDataSource, so offline UI development exercises the real video-domain contract.

Closure callbacks capture [weak self]. That avoids a cycle between the coordinator and long-lived input helpers. Callback bodies mutate main-actor state because the helpers themselves are main-actor isolated.

Make lifecycle cancellation explicit

SOURCE TRAIL – ANALYZING NOW: `RobotViewModel.start()`, `stop()`, `setSceneActive(_:)`, and `deinit`.

The controller owns several independent tasks: session updates, connect/disconnect actions, virtual input refresh, video actions, and video-pipeline lifecycle. Each task has a property and an identity when stale completion could race a replacement.

`start()` is idempotent: it returns if the updates task already exists. It starts physical input providers, obtains the session's asynchronous update stream, and iterates with cancellation checks. `stop()` cancels every owned task, invalidates motion, stops video and speech, stops hardware input, and asynchronously disconnects the session.

The scene phase is a safety input. When inactive, the view model clears the uninstalled pairing draft, cancels video actions, ends virtual motion, stops decoding, tells the session that the scene is inactive, and attempts to unsubscribe. Returning active does not silently arm motion.

Use a lifecycle generation or action UUID when cancellation alone is insufficient. A cancelled operation may still unwind later. The completion method checks identity before clearing the current task slot:

```
1 private func finishAction(_ id: UUID) {  
2     guard actionID == id else { return }  
3     actionID = nil  
4     actionTask = nil  
5 }
```

This is a compact defense against an old task erasing ownership of a newer one.

Put the state machine in an actor

SOURCE TRAIL – ANALYZING NOW: `RobotSession.swift` and `ConnectionModels.swift` in the package's `Connection` directory.

`RobotSession` owns connection phase, handshake, command sequencing, safety evaluation, pending video continuations, timeouts, open media-channel ownership, and the asynchronous snapshot stream. An actor serializes these mutations without forcing network and media work onto the main actor.

The view model never edits a connection snapshot directly. It asks the actor to connect, disconnect, arm, stop, update input, or subscribe. The actor publishes immutable snapshots. This is unidirectional data flow:

```

1 view intent -> RobotViewModel -> RobotSession actor -> RobotTransport
2                                     |
3                                     v
4                               AsyncStream snapshots
5                                     |
6                                     v
7                               main-actor presentation

```

Pending subscription calls use checked continuations, indexed by subscription ID, with independent timeout tasks. Cancellation removes abandoned IDs and prevents a late reply from resurrecting an abandoned operation. Open data channels remember which transport owns them so teardown calls the correct adapter.

When adding a new request/reply operation, copy this discipline: unique request ID, exactly one continuation, deadline, cancellation handler, late-reply rejection, disconnect cleanup, and tests for every ordering.

Define a domain protocol before a wire format

SOURCE TRAIL – ANALYZING NOW: `Control/ControlProtocol.swift`, `Video/VideoProtocol.swift`, `Video/VideoDataTransport.swift`, and `CerebroRobotTransport.swift`.

`RobotTransport` is a `Sendable` protocol with `connect`, `disconnect`, `send`, and `events` operations. Application and session code depend on it instead of `Network.framework`.

The command domain includes arming, drive, stop, emergency stop, reset, video messages, and operator text. `RobotCommandEnvelope` carries protocol version, session UUID, monotonically increasing sequence, issue time, lease duration, and command. These fields make freshness and session binding explicit.

Numeric value objects sanitize at construction. `MotionVector`, `CameraVector`, and `TorsoVector` convert non-finite values to zero and clamp finite values to -1 through 1. `ControllerPose` rejects non-finite data, negative timestamps, and implausible quaternion magnitude, then normalizes the quaternion.

This technique prevents invalid floating-point values from spreading, but it should not silently conceal every programming error. Add debug assertions or diagnostics at trusted call sites when sanitization occurs.

Codable enums use an explicit type discriminator and associated payload keys. This is more stable and reviewable than relying on synthesized representation for a protocol shared between repositories.

Treat dead-man logic as a pure decision function

SOURCE TRAIL – ANALYZING NOW: `DeadManController.swift` and its matching tests in the package. The source map prints their full paths.

`DeadManController` is a value type. It remembers armed state, latched emergency stop, scene activity, latest sequenced sample, receipt time, and a forced inhibit reason. Its default input timeout is 250 milliseconds.

Evaluation follows a deliberate precedence:

1. emergency stop;
2. connection readiness;
3. active scene;
4. armed state;
5. forced inhibit;
6. presence of input;
7. held dead-man state;
8. input age;
9. drive decision.

That ordering makes failures deterministic. A diagnostic controller pose may survive into a stop decision, but motion, camera, gripper, and torso authority do not.

Arming does not manufacture a fresh input. Resetting emergency stop disarms. A new sample must have a greater sequence than the retained sample. Use `ContinuousClock`, not wall-clock time, for local lease age because wall time can jump.

Tests should cover equality at the deadline, stale sequence, NaN clamping, disconnect, scene transitions, arming while E-stop is latched, reset behavior, and release.

Translate controller hardware into one sample

SOURCE TRAIL – ANALYZING NOW: Platform/GameController/GameControllerInput.swift and Resources/Info.plist.

The app declares spatial and extended gamepad profiles. `GameControllerInput` observes connect/disconnect notifications, configures existing devices, starts wireless discovery, and stores state by object identity.

A conventional extended gamepad controls both treads. The left and right thumbstick Y axes remain independent tank-drive demands. The right thumbstick also supplies camera axes for compatibility. A conventional dead-man is **A or both shoulder buttons**. Index triggers remain reserved for grippers.

PSVR Sense-style spatial controllers arrive as `GCPhysicalInputProfile` rather than `GCExtendedGamepad`. The first supported one is assigned left, the next right. Singular and left/right element names are probed because SDK and controller mappings differ. Each controller's grip is the hold gesture; its index trigger becomes that side's Boolean gripper request.

Two VR grips must be held in the combined sample. Releasing either removes the continuous dead-man. Trigger values use a threshold above 0.5. Gripper commands are desired open/closed states, not measured jaw position.

Callbacks are not trusted as an everlasting stream. The implementation also polls `lastEventTimestamp` every 50 milliseconds and processes only a strictly newer timestamp. Missing fresh callbacks must lead to expiry in the session rather than replaying retained stick values.

Combine two controllers carefully

Per-device state contains side, last timestamp, primary stick, second stick, camera values, optional pose, grip, and trigger. Combination must be independent of dictionary iteration order. Left and right sides populate their corresponding tread, trigger, and pose. The aggregate `deadManIsHeld` requires the intended hardware combination.

When controller mappings change, add fixture logging on a development build, update the lookup table, and keep unknown element names out of production logs if they reveal device details.

Convert tank drive without losing information

SOURCE TRAIL – ANALYZING NOW: `RobotViewModel.sendCombinedControllerSample()`.

The wire domain uses linear and angular motion while VR sticks produce left and right tread values. The exact reversible conversion used by the app is:

```
1 let linear = (leftTread + rightTread) * 0.5
2 let angular = (leftTread - rightTread) * 0.5
```

The receiver can reconstruct left as `linear + angular` and right as `linear - angular`, subject to later limiting. Multiplying the two resulting components by one speed limit preserves their relationship.

Every combined sample increments a wrapping `UInt64` sequence, declares `.gameController`, includes head-derived camera and torso values, carries gripper Booleans and optional controller poses, and repeats the dead-man state. The session—not the input class—decides whether the sample has authority.

Track head orientation relative to consent

SOURCE TRAIL – ANALYZING NOW: Platform/HeadOrientationInput.swift.

HeadOrientationInput owns an ARKitSession and WorldTrackingProvider. It polls a device anchor every 40 milliseconds using CACurrentMediaTime. It does nothing unless the dead-man is held.

The first tracked orientation after the hold begins becomes the baseline. Relative orientation is:

```
1 let relative = baseline.inverse * orientation
2 let forward = simd_normalize(relative.act(SIMD3(0, 0, -1)))
3 let yaw = atan2(-forward.x, -forward.z)
4 let pitch = asin(max(-1, min(1, forward.y)))
```

Yaw maps to full camera pan at ± 60 degrees. Pitch maps to full camera tilt at ± 35 degrees. Both normalize to -1 through 1. Torso rotation remains zero inside the neck range; excess yaw between 60 and 180 degrees maps to normalized torso demand. Cerebro owns the Pololu Tic safety and physical position conversion.

Releasing the dead-man clears the baseline and emits unavailable. Tracking loss while held also emits unavailable. This prevents an old head pose from remaining active. Re-engaging naturally recenters at the current viewing direction.

Coordinate-frame bugs are common here. Record whether a transform maps anchor to origin or origin to anchor, define forward, and test signed yaw and pitch with known poses. Never “fix” one direction by adding unexplained negations downstream.

Understand controller pose as telemetry, not authority

SOURCE TRAIL – ANALYZING NOW: GameControllerInput.swift, ControlProtocol.swift, and TelemetryPanel.swift.

Spatial controller poses are normalized, timestamped data included in a control sample and reflected in diagnostics. They do not bypass arming, dead-man, freshness, collision policy, or the robot's physical stop.

The position and quaternion describe a pose in the framework's tracking frame. They are not automatically an AMBER arm joint solution. Turning pose into arm motion requires calibrated transforms, workspace constraints, inverse kinematics, self/robot/environment collision checks, rate limiting, joint feedback, and cancellation behavior.

Keep raw pose, calibrated target, planned joint state, commanded joint state, and measured joint state as separate types. A single seven-float array cannot express their different truth claims.

Design press-and-hold spatial controls

SOURCE TRAIL – ANALYZING NOW: `ControlPanel.swift` and the `beginVirtualMotion` method in `RobotViewModel.swift`.

The UI alternative to a gamepad uses press-and-hold controls. Beginning a hold checks active scene, ready connection, armed state, and unlatched emergency stop. It captures speed-limited linear/angular values and starts an 80-millisecond refresh task. Ending cancels the task and explicitly invalidates input as dead-man released.

Do not implement motion as a one-shot button tap followed by a timer on the robot. The client should continuously renew a short lease, and the receiver should independently stop when renewal ceases.

SwiftUI gestures can end through release, cancellation, focus change, view removal, scene inactivity, or application termination. Route every observable end into the same invalidation method. Still assume the client can disappear without calling it; the robot-side watchdog is mandatory.

Pair identity, not merely an address

SOURCE TRAIL – ANALYZING NOW: the credential, pairing-store, pinned-TLS, control-discovery, and pairing-sheet files. Exact names and paths are in the source map.

The user pastes a complete R0BCTL2: enrollment code. Installation parses and stores a unique controller credential plus Cerebro certificate pin in Keychain. "Credential installed" means local storage succeeded; only a live pinned-TLS and reciprocal-proof exchange earns "Connected and verified."

Bonjour _robctl._udp discovery filters for the paired robot identity. Each physical controller needs a distinct credential so it can be revoked and duplicate sessions can be detected. Never copy another controller's Keychain item.

Pinned TLS prevents a same-LAN service with another certificate from silently impersonating Cerebro. Reciprocal HMAC proof binds both peers to the pairing secret and role. The accepted handshake establishes the exact live session UUID used in subsequent envelopes and video authorization.

Certificate rotation must be intentional. An unexpected pin change should fail closed; deleting pin checks to make a demo connect converts an availability problem into an authentication vulnerability.

Separate control and video failure domains

SOURCE TRAIL – ANALYZING NOW: `CerebroRobotTransport.swift`, the control client, the video client, and both discovery files.

Control uses `_robctl._udp` with application protocol `robctl/2`. Video uses `_robvideo._udp` with `robvideo/1`. Both are QUIC/TLS services, but they have distinct discovery, authentication exchanges, clients, framing, and lifecycle.

The authenticated control connection creates the live session UUID. Video proof and subscription carry that exact UUID. Disconnecting or replacing control, revoking credentials, suspending the scene, or unsubscribing tears down video. Video discovery or decode failure must not disconnect authenticated motion control.

This separation prevents megabytes of media and decoder backpressure from delaying a stop command. It also keeps optional camera availability from becoming a prerequisite for control.

Negotiate video before allocating a decoder

SOURCE TRAIL – ANALYZING NOW: `Video/VideoProtocol.swift`, `ROBVideoWireTypes.swift`, and `RobotViewModel.videoRequest`.

After authentication, Cerebro sends bounded camera capabilities. The client requests a camera ID, preferred codecs, dimensions, frame rate, bitrate, and delivery mode. The current production request is H.264, at most 960 by 540, 20 frames per second, and 1.5 Mbit/s. Production selects `reliableStream`; the synthetic simulator uses its in-memory datagram-shaped path.

The wire mapper rejects empty or oversized identifiers, duplicate cameras, zero dimensions, limits above hard caps, missing H.264, missing reliable-stream support, unknown JSON keys, and unsupported delivery combinations.

Only an accepted response creates a `VideoStreamDescriptor`. The descriptor carries session, subscription, codec, geometry, rate, bitrate, and delivery facts used to validate subsequent media.

Subscription code uses a unique ID and a continuation. The session rejects duplicate IDs, times out the request, handles cancellation, and remembers abandoned IDs so a late response cannot create an unwanted stream.

Frame ordered QUIC video defensively

SOURCE TRAIL – ANALYZING NOW: `Video/ROBVideoFramer.swift`, `ROBVideoWireTypes.swift`, and `ROBVideoAuthentication.swift`.

The ordered stream uses a fixed 32-byte big-endian RVID header:

Offset	Size	Field
0	4	magic RVID
4	1	protocol version
5	1	header size
6	2	message type
8	4	payload length
12	4	reserved zeros
16	8	increasing sequence
24	8	reserved zeros

The framer rejects invalid magic, version, size, type, reserved bits, excessive payload, and non-increasing input sequence. Control JSON is capped at 64 KiB. Codec configuration is capped at 64 KiB. Access units are capped at 2 MiB, and total framed media has a derived maximum.

The Network framework framer separates message boundaries from stream reads. Its parser waits for an exact header; no-copy delivery emits one validated payload with metadata. Output writes the header and payload without copying when possible.

Sequence protects stream logic from duplicate or reordered application frames; TLS provides confidentiality and integrity. It is not a replacement for the session/stream/access-unit sequence validation inside the media domain.

Carry codec configuration and access units explicitly

SOURCE TRAIL – ANALYZING NOW: `EncodedVideoProtocol.swift`, the H.264 sample-buffer factory, and the H.264 receiver.

H.264 decoding needs parameter sets and access units. The protocol sends codec configuration separately with SPS/PPS bytes, NAL length-field size, generation, session ID, and stream ID. Access units carry sequence, presentation timestamp, timescale, duration, keyframe flag, and AVCC payload.

`VideoStreamValidator` binds every message to the negotiated session and stream, checks configuration generation, sequence, timestamps, dimensions, and keyframe recovery state. A gap makes predictive frames unsafe because they may depend on a missing reference. The receiver drops them, requires a new keyframe, and sends bounded feedback.

The H.264 sample-buffer factory creates a Core Media video-format description from parameter sets and constructs timed sample buffers from AVCC payload. It validates expected dimensions rather than trusting the bitstream to allocate arbitrary surfaces.

Never assume Annex B start codes and AVCC length-prefixed NAL units are interchangeable. Convert at one named boundary and test both malformed and valid fixtures.

Decode without blocking control or the main actor

SOURCE TRAIL – ANALYZING NOW: the H.264 receiver in the video library and the pipeline coordinator in the application. Their exact filenames are in the source map.

`H264VideoReceiver` is an actor. It consumes an `AsyncThrowingStream`, updates statistics, validates messages, configures the sample-buffer factory, and enqueues accepted frames. Decoder pressure never waits indefinitely: renderer readiness gets a 50-millisecond bounded wait with two-millisecond cancellation-aware polling.

When the renderer fails or requires a flush, the receiver drops the access unit, flushes, requires a keyframe, and requests one. Keyframe requests are rate-limited to one per 500 milliseconds during continuing recovery. Statistics publish at most twice per second unless forced.

`AVSampleBufferVideoRenderer` is documented for background enqueueing, but its SDK annotation does not satisfy Swift 6 isolation when obtained from a main-actor display layer. The code uses a narrow `@unchecked Sendable` handle that records this framework guarantee. Do not spread `@unchecked Sendable` across the display layer or arbitrary UIKit objects; isolate the exception and explain it.

`VideoPipelineCoordinator` remains `@MainActor` because it owns display state. A lifecycle generation and pipeline UUID prevent an old open/close operation from taking over a replacement pipeline. It closes an unowned channel on every failed handoff, avoiding media-channel leaks.

Present AVFoundation media inside SwiftUI

SOURCE TRAIL – ANALYZING NOW: `SampleBufferVideoView.swift` and `VideoPanel.swift`.

`SampleBufferVideoView` is a `UIViewRepresentable`. Its host view owns no decoder; it attaches the coordinator's `AVSampleBufferDisplayLayer`, removes any prior layer, uses `.resizeAspect`, and updates the layer frame in `layoutSubviews` with implicit animations disabled.

`dismantleUIView` detaches the display layer. This matters when SwiftUI reconstructs or removes the representable. A display layer must not remain attached to two superlayers.

`VideoPanel` treats availability, subscription, pipeline state, and statistics as different concepts. A connected control session may have no camera. A camera may exist with no subscription. A subscription may be accepted while decoding is starting or failed.

Keep the camera large and visually primary, but do not overlay controls that can hide connection or safety state.

Understand audio: capture locally, transmit text

SOURCE TRAIL – ANALYZING NOW: `VisionSpeechInput.swift`, `OperatorSpeechPanel.swift`, the view model's `text-send` method, and `ControlProtocol.swift`.

The current Vision Pro controller does **not** stream microphone audio to Cerebro. Its audio path is:

```
1 Vision Pro microphone
2   -> AVAudioEngine input node
3   -> 1024-frame tap
4   -> SFSpeechAudioBufferRecognitionRequest
5   -> SFSpeechRecognizer
6   -> partial/final transcript
7   -> editable SwiftUI String
8   -> OperatorTextMessage over authenticated control
```

The input first requests Speech authorization, then microphone permission. It creates a recognizer for the current locale, verifies availability, enables partial results, and requires on-device recognition when the device supports it:

```
1 request.shouldReportPartialResults = true
2 request.requiresOnDeviceRecognition = recognizer.supportsOnDeviceRecognition
```

That assignment does not promise every locale or OS state can recognize offline; it requests on-device operation when supported. The UI must report unavailability rather than silently changing privacy behavior.

The `AVAudioEngine` input node installs a bus-zero tap with a 1024-frame buffer and the node's output format. Each buffer is appended to the recognition request. `prepare()` and `start()` may throw. Every failure and stop path removes the tap, ends/cancels recognition, clears retained objects, and updates UI state.

Recognition callbacks enter a `Task { @MainActor ... }` before touching observable state. Partial transcripts update the draft. A final transcript invokes the callback with `isFinal`, updates status, and stops capture.

Command versus puppet speech

`OperatorTextMode.command` asks Cerebro to process text like its local text input. `puppetSpeech` asks ROB to speak the text verbatim without sending it to the AI or motion parser. Both modes carry a Codable `OperatorTextMessage`; neither arms motion.

The current initializer automatically sends a final dictation using the mode selected at completion time. The panel also offers explicit `Send as Input` and `ROB Says It` buttons. For higher-consequence installations, consider changing auto-send to review-before-send, especially when ambient speech may be misrecognized.

If true audio streaming is added later

Treat it as a new protocol, not an extra field in operator text. Specify format, sample rate, channel count, frames per packet, timestamps, clock domain, jitter buffer, congestion/backpressure policy, echo cancellation, interruption handling, authentication, consent indication, retention, and receiver ownership. Keep it off the motion channel. Add `Info.plist` disclosure and an unmistakable live microphone indicator.

Declare privacy and device capabilities

SOURCE TRAIL – ANALYZING NOW: Resources/Info.plist.

The plist declares:

- local-network usage for robot discovery;
- Bonjour service types `_robot1._udp` and `_robvideo._udp`;
- microphone use for explicit dictation;
- speech recognition for conversion to reviewable text;
- accessory tracking for spatial-controller pose;
- spatial and extended game-controller profiles;
- one window scene.

Usage descriptions should say what data is used for and when. They are user communication, not mere App Store obstacles. Adding raw audio transmission would make the present wording incomplete.

Test permission states independently: not determined, denied, restricted, authorized, recognizer unavailable, microphone interruption, local network denied, and accessory tracking unavailable.

Design failures as ordinary states

SOURCE TRAIL – ANALYZING NOW: the connection-status view, telemetry panel, Cerebro transport errors, and video pipeline errors.

The UI should distinguish disconnected, connecting, handshaking, connected, disconnecting, and failed. Installed pairing is not verified pairing. Video unavailable is not control unavailable. Speech unavailable is not motion failure. Tracking unavailable must neutralize camera/torso demand without inventing a pose.

Prefer typed error cases with `LocalizedError` descriptions at presentation boundaries. Avoid logging secrets, pairing codes, HMAC material, certificate private keys, or raw dictated content by default.

Status strings help a human, but state must remain typed. Do not make button enablement depend on parsing a status message.

Test the package in layers

SOURCE TRAIL – ANALYZING NOW: all files under `Packages/ROBControlCore/Tests`.

The package includes:

- dead-man transition and timeout tests;
- simulator connection and watchdog tests;
- video-domain Codable and validation tests;
- synthetic pixel-buffer and end-to-end pipeline tests;
- legacy payload compatibility tests;
- control wire, pairing, pinning, and authentication tests;
- video wire compatibility tests.

Write pure tests for value normalization and state machines first. Use actors and asynchronous streams in integration tests. Give every async expectation a deadline. Explicitly cancel tasks and close streams so a passing test does not leak work into the next test.

The simulator is a protocol peer, not a visual mock. It independently evaluates input age, integrates a deterministic pose, reports telemetry, negotiates video, and serves synthetic H.264 through the same domain boundaries.

Package tests cannot validate visionOS focus delivery, actual controllers, ARKit tracking, Local Network permission, Bonjour on a LAN, Keychain entitlements, camera capture on Cerebro, decoder behavior on device, or physical stopping. Maintain a separate two-device checklist.

Debug without drowning Xcode

SOURCE TRAIL – ANALYZING NOW: README.md diagnostics plus the relevant transport state handlers.

Use focused evidence:

```
1 dns-sd -B _robotl._udp local.  
2 dns-sd -B _robvideo._udp local.
```

Then observe one layer at a time: controller connection and timestamp, app focus, dead-man sample, session decision, control handshake, command acknowledgement, video authentication, subscription, codec configuration, access-unit sequence, render statistics.

Rate-limit repetitive logs. The earlier “bad file descriptor” style flood can make the debugger unusable and hide the first failure. Log state transitions and counters, not every poll. Use unified logging categories with privacy annotations in a future hardening pass.

Add a feature without breaking authority

SOURCE TRAIL – ANALYZING NOW: use the whole chain for the nearest existing feature before editing.

For a new controller gesture:

1. identify the physical element and supported profiles;
2. add per-device state without changing dead-man semantics;
3. combine deterministically;
4. add a bounded domain type;
5. include it in the control sample and decision;
6. encode it in an explicit protocol field;
7. update the matching Cerebro decoder;
8. keep execution behind robot-side state and limits;
9. add fixture and state-machine tests;
10. test simulator, then two devices, then restrained hardware.

For a new video codec, update capability negotiation, hard limits, wire representation, configuration records, decoder factory, recovery rules, fixtures, and robot encoder. Do not select a codec merely because both platforms expose a framework enum.

For a new speech mode, add a Codable enum case, deterministic Cerebro routing, UI explanation, backward-compatibility behavior, and tests proving it cannot arm or create raw motion.

Swift techniques worth carrying elsewhere

The codebase demonstrates several reusable techniques:

- `@MainActor @Observable final` class for presentation coordination;
- `@ObservationIgnored` for internal tasks and services;
- actors for mutable session and media state;
- immutable `Sendable` value objects at boundaries;
- explicit clamping and finite-number validation;
- dependency inversion through transport protocols;
- `AsyncStream` for state/event delivery;
- checked continuations with cancellation and deadlines;
- generation tokens against stale async completion;
- weak callback capture against ownership cycles;
- `ContinuousClock` for leases and local timeouts;
- narrow, documented `@unchecked Sendable` framework adapters;
- `UIViewRepresentable` for a carefully owned `AVFoundation` surface;
- separate optional media and authoritative control failure domains;
- explicit protocol versions, discriminators, limits, reserved fields, and sequence numbers.

None of these techniques is automatically correct. Their value comes from matching ownership and failure behavior to the real system.

Production review checklist

- ☐ package tests pass under the committed Swift toolchain;
- ☐ visionOS simulator target builds without signing;
- ☐ bundle ID, team, and entitlements are intentionally configured;
- ☐ every device has a unique pairing credential;
- ☐ certificate pin and reciprocal proof fail closed;
- ☐ app focus and scene inactivity invalidate input;
- ☐ 250 ms client lease and Cerebro watchdog are measured together;
- ☐ controller disconnect and missing callbacks stop motion;
- ☐ both VR grips implement the intended dead-man gesture;
- ☐ index triggers affect only the matching gripper request;
- ☐ head baseline resets on every hold and tracking loss neutralizes output;
- ☐ torso rotation begins only outside the neck range;
- ☐ video failure never delays or disconnects control;
- ☐ video messages are session/stream/sequence bounded;
- ☐ decoder pressure drops or recovers media rather than blocking;
- ☐ microphone and speech permissions are understandable;
- ☐ documentation states that microphone audio is local and only text is transmitted;
- ☐ command and puppet-speech routes remain distinct;
- ☐ logs exclude secrets and dictated content by default;
- ☐ simulator, two-device, and restrained-hardware tests are recorded;
- ☐ physical emergency stop behavior is independently verified.

File-by-file reading order

Use this order for a new Swift contributor:

1. README.md
2. Packages/ROBControlCore/Package.swift
3. Control/ControlProtocol.swift
4. Control/DeadManController.swift
5. Connection/RobotSession.swift
6. Simulation/SimulatedRobotEndpoint.swift
7. App/RobotViewModel.swift
8. App/ContentView.swift
9. Platform/GameController/GameControllerInput.swift
10. Platform/HeadOrientationInput.swift
11. Platform/VisionSpeechInput.swift
12. ROBCerebroTransport/CerebroRobotTransport.swift
13. control discovery, wire, client, and pairing files
14. Video/VideoProtocol.swift and EncodedVideoProtocol.swift
15. video discovery, authentication, framer, wire types, and client
16. H264VideoReceiver.swift and H264SampleBufferFactory.swift
17. VideoPipelineCoordinator.swift
18. SampleBufferVideoView.swift and VideoPanel.swift
19. feature panels
20. every corresponding test before making a change

When a shortened path is ambiguous, use `OPEN-SOURCE-CODE-MAP.md` or find it from the workspace root:

```
1 rg --files ROBControllerVision | rg 'DeadManController|VisionSpeechInput|ROBVideoFramer'
```

Closing principle

ROBControllerVision is compelling because Vision Pro can combine a large first-person camera, spatial controls, tracked head orientation, controller pose, dictation, and a wraparound instrument deck. It is trustworthy only when those rich inputs are translated into small, typed, bounded, expiring requests.

The most important Swift skill in this controller is not a particular modifier or framework call. It is making ownership visible: which actor owns state, which task owns work, which session owns a message, which view owns a layer, which controller owns an input, which permission owns capture, which receiver owns a timeout, and which physical mechanism remains outside the app's authority.